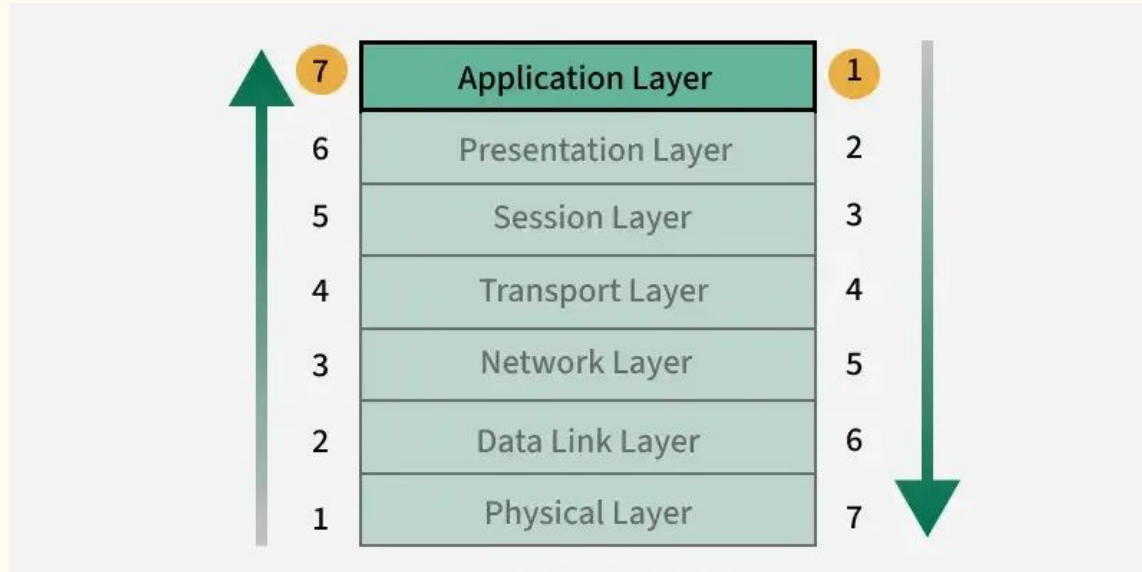


Application Layer

Er. Sudarshan Devkota

Application Layer

- The Application Layer is the topmost layer of the OSI model and serves as the direct interface between the user and the network.
- It does not provide services to other layers but instead provides services to the software applications that reside outside the network model.



Functions of Application Layer

1. Data Representation and Translation

- One of the primary responsibilities of this layer is ensuring that data is presented in a format that both the sender and receiver can accurately interpret.
- It converts user-entered information into a standard, network-compatible format before transmission and interprets received data back into a user-friendly form.
- This includes character encoding (e.g., converting text to ASCII) and multimedia compression or formatting (e.g., JPEG for images or HTML for web content).

Functions of Application Layer

2. Network Service Access

The Application Layer serves as the gateway for software to interact with the underlying network infrastructure. It provides the necessary hooks and APIs that allow user applications to access network-wide services.

It enables high-level applications such as email clients, web browsers, and file managers—to request network resources.



File Transfer



Web Surfing



Emails

Functions of Application Layer

3. Application Protocols and Standardization

It defines a rigorous set of rules and procedures that applications must follow. These rules govern how messages are formatted, how data is exchanged, and how service requests are initiated.

Protocols define the specific syntax and semantics of the data being sent (e.g., the structure of an HTTP request).



Functions of Application Layer

4. Session Management and Synchronization

The Application Layer is responsible for the lifecycle of a communication "session" the period during which two applications are actively exchanging data.

It handles the three distinct phases of a session: Establishment (setting up the connection), Management (maintaining the flow and synchronization), and Termination (gracefully closing the connection).

It ensures that if a long communication is interrupted, the applications can recover or restart from a known checkpoint.



Application Layer Protocols: WWW

The World Wide Web (WWW) is a distributed information system that operates as an application on top of the Internet. It is a collection of interconnected documents and other web resources, linked by hyperlinks and URLs.

The Content: Resources on the web are primarily composed of HTML documents, but also include images, videos, and scripts.

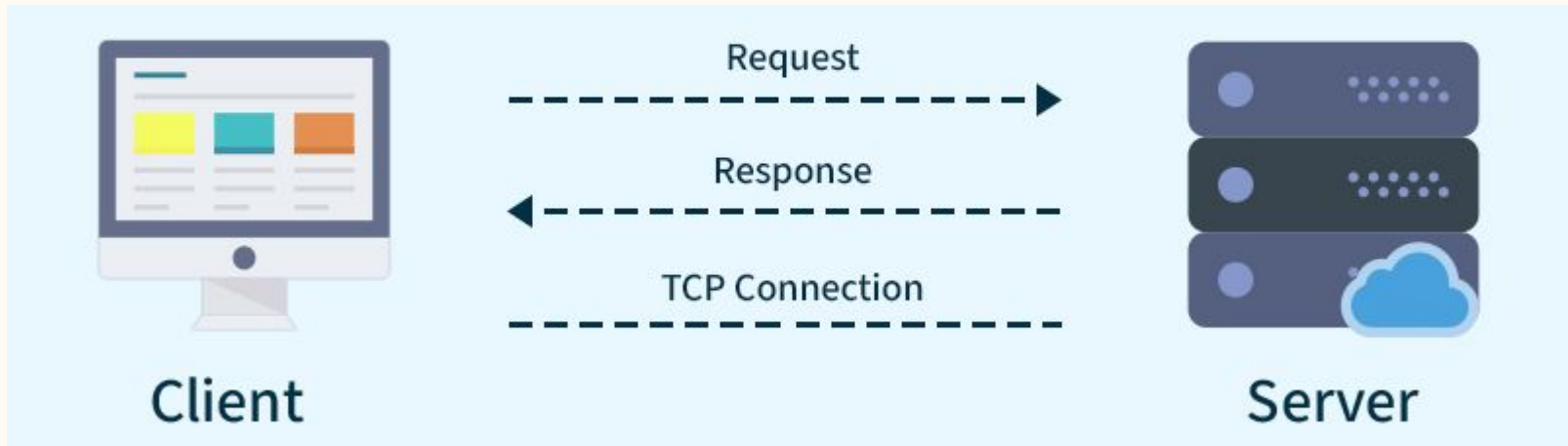
The Framework: It provides a standardized way for users to access, retrieve, and manage files across a global network.

The Identity: Every piece of information on the web is identified by a Uniform Resource Locator (URL), which acts as the global address for that specific resource.

Application Layer Protocols: HTTP

The Hypertext Transfer Protocol (HTTP) is the primary protocol used by the World Wide Web to define how messages are formatted and transmitted, and what actions web servers and browsers should take in response to various command

It sits on top of TCP (Transmission Control Protocol) to ensure that the data of a webpage is delivered accurately and in the correct order.



Application Layer Protocols: HTTP

Core Characteristics:

Stateless Protocol: By design, HTTP is stateless. This means the server does not "remember" previous requests from the same client. Each request is treated as an independent transaction. (Note: Technologies like Cookies are used at the Application Layer to provide statefulness, such as keeping a user logged in).

Connectionless (Original Design): The client initiates a request, and after the server responds, the connection is disconnected. Modern versions (HTTP/1.1 and later) use "Persistent Connections" to allow multiple requests over a single TCP connection to improve speed.

Media Independence: HTTP can handle any type of data as long as both the client and server know how to handle the data content (MIME types).

Application Layer Protocols: HTTP

HTTP communication always follows a request-response pattern between a Client (User Agent/Browser) and a Server (Web Server).

A. The HTTP Request:

When you click a link or type a URL, the browser sends a request message consisting of:

Request Line: Includes the Method (e.g., GET, POST), the URL, and the HTTP version.

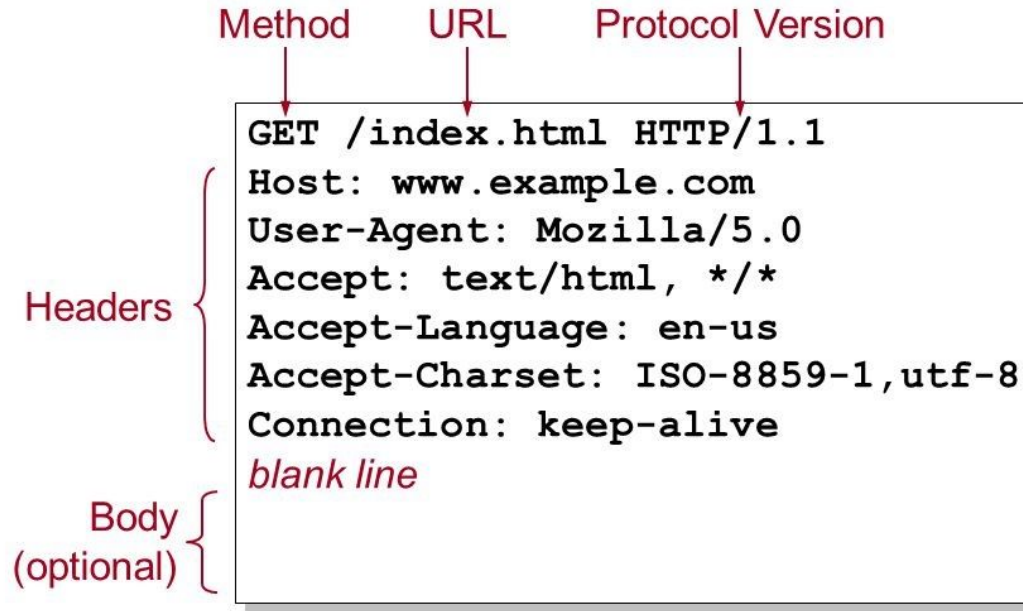
Request Headers: Metadata about the browser, such as User-Agent, Accept-Language, and Host.

Empty Line: Separates headers from the body.

Message Body: (Optional) Contains data sent to the server, commonly used in POST requests (like submitting a form).

Application Layer Protocols: HTTP

HTTP Request



The diagram illustrates the structure of an HTTP Request. It shows a sequence of components: a line with the Method, URL, and Protocol Version; a block of Headers; a blank line; and an optional Body. Arrows point from the labels 'Method', 'URL', and 'Protocol Version' to their respective parts in the request line. A bracket on the left groups the header lines under the label 'Headers'. Another bracket on the left groups the empty space under the label 'Body (optional)'.

Method URL Protocol Version

GET /index.html HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html, */*

Accept-Language: en-us

Accept-Charset: ISO-8859-1,utf-8

Connection: keep-alive

blank line

Body
(optional)

Application Layer Protocols: HTTP

B. The HTTP Response:

The server processes the request and sends back a response:

Status Line: Includes the HTTP version and a Status Code (e.g., 200 OK, 404 Not Found).

Response Headers: Metadata about the server and the data being sent (e.g., Content-Type: text/html, Server: Apache).

Empty Line: Separates headers from the body.

Message Body: The actual content of the resource (the HTML code, an image file, etc.).

Application Layer Protocols: HTTP

RESPONSE

```
HTTP/1.1 200 OK
```

HTTP response
status line

```
Date: Wed, 06 Jul 2022 09:30:28 GMT  
Accept-Ranges: bytes  
Content-Length: 2005  
Content-Type: text/css; charset=UTF-8  
<CRLF>
```

HTTP response
headers

```
nav.navbar {  
    ...some style  
}
```

HTTP response
body

Application Layer Protocols: HTTP

Common HTTP Methods

Methods define the specific action the client wants the server to perform:

GET: Requests a representation of the specified resource. This should only retrieve data.

POST: Used to send data to the server to create or update a resource (e.g., submitting a username and password).

PUT: Replaces all current representations of the target resource with the request payload.

DELETE: Deletes the specified resource on the server.

Application Layer Protocols: HTTP

Understanding HTTP Status Codes:

Status codes are three-digit integers where the first digit defines the class of the response:

1xx (*Informational*): Request received, continuing process.

2xx (*Success*): The action was successfully received, understood, and accepted (e.g., 200 OK).

3xx (*Redirection*): Further action needs to be taken to complete the request (e.g., 301 Moved Permanently).

4xx (*Client Error*): The request contains bad syntax or cannot be fulfilled (e.g., 404 Not Found).

5xx (*Server Error*): The server failed to fulfill an apparently valid request (e.g., 500 Internal Server Error).

Application Layer Protocols: HTTPS

HTTPS is not a separate protocol from HTTP; rather, it refers to the use of HTTP over a secure channel.

- It utilizes SSL (Secure Sockets Layer) or its more modern and secure successor, TLS (Transport Layer Security).
- All data, including URLs, query parameters, headers, and cookies, is encrypted before it leaves the device.
- While HTTP operates on Port 80, HTTPS exclusively uses Port 443.

Application Layer Protocols: HTTPS

SSL/TLS Certificates and Digital Identity:

For HTTPS to work, a website must have an SSL Certificate installed on its server. This certificate acts as a "digital passport."

What is in the Certificate?

It contains the website's public key, the domain name, the expiration date, and the digital signature of the entity that issued it.

The Role of the Certificate?

It allows the browser to verify that the web server actually belongs to the domain it claims to represent.

Application Layer Protocols: HTTPS

Certificate Authorities (CA): The "Trusted Third Party"

A CA is a trusted organization (like DigiCert, Let's Encrypt, or GlobalSign) that verifies the identity of a website owner before issuing a certificate.

Your web browser comes pre-installed with a list of "Root Certificates" from trusted CAs.

When you visit a site, the browser checks if the site's certificate was signed by one of these trusted authorities.

If a website uses a "self-signed" certificate or one from an unknown CA, the browser will display a security warning because the identity of the server cannot be independently verified.

Application Layer Protocols: HTTPS

How HTTPS Establishes a Connection

Client Hello: The browser sends its supported encryption versions to the server.

Server Hello & Certificate: The server sends its SSL certificate (containing its public key) to the browser.

Verification: The browser checks the certificate against its list of trusted CAs to ensure it is valid and authentic.

Key Exchange: The browser and server use the public key to agree on a "Session Key" a unique, temporary symmetric key used only for this specific session.

Secure Transmission: Once the handshake is complete, all further communication is encrypted using the session key.

Application Layer Protocols: DNS

The Domain Name System (DNS) is a fundamental hierarchical and decentralized naming system used to identify computers, services, and other resources reachable through the Internet or other Internet Protocol (IP) networks.

- In a network environment, hardware communicates using numerical IP addresses (e.g., 192.0.2.1 or 2001:db8::1). However, humans find it difficult to memorize long strings of numbers.
- DNS translates human-readable domain names (like `www.google.com`) into machine-readable IP addresses.
- This process happens entirely in the background. When you type a URL into a browser, the Application Layer triggers a DNS query before any HTTP communication begins.

Application Layer Protocols: DNS

The Hierarchical Structure of a Domain Name:

DNS is organized into an inverted tree structure, ensuring that no single server has to store every name on the Internet.

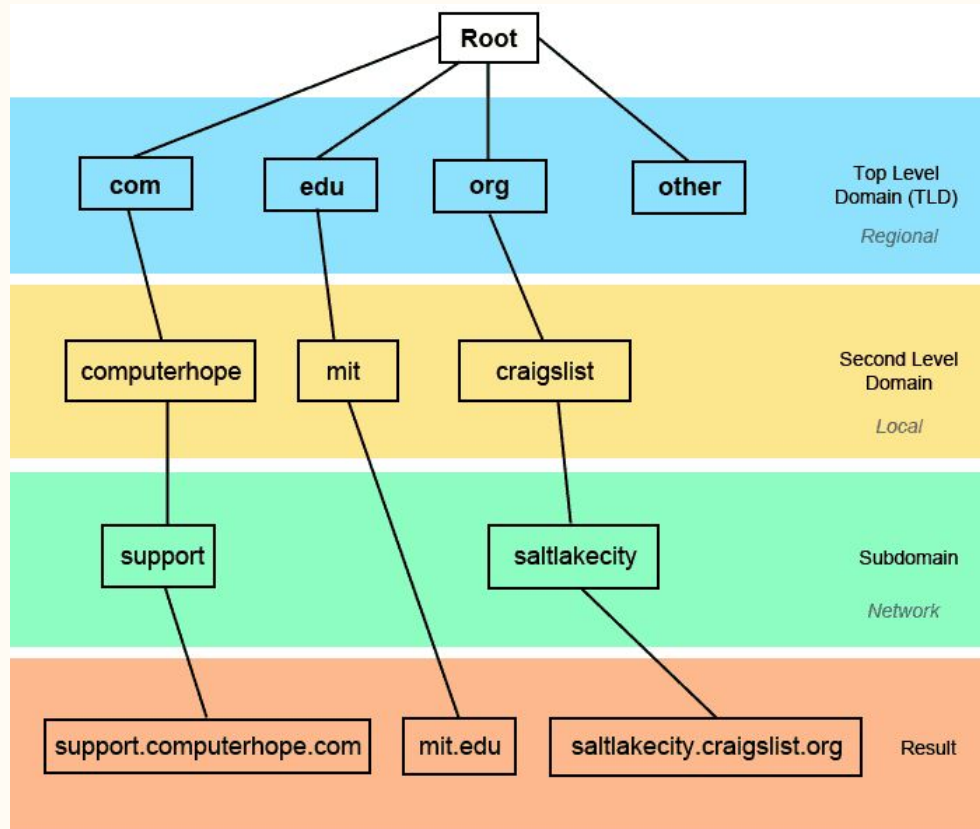
Root Level: Represented by a "dot" (implicitly at the end of every domain, e.g., google.com.). Root servers direct queries to the appropriate Top-Level Domain (TLD) server.

Top-Level Domain (TLD): These are the suffixes we recognize, such as .com, .org, .edu, and country codes like .np or .uk.

Second-Level Domain: This is the actual name of the entity, such as google, wikipedia, or tribhuvan-university.

Subdomain: Specific departments or services within a domain, such as mail.google.com or it.tu.edu.np.

Application Layer Protocols: DNS



Application Layer Protocols: DNS

How DNS Resolution Works:

The resolution process involves several "nameservers" working in a sequence to find the correct address.

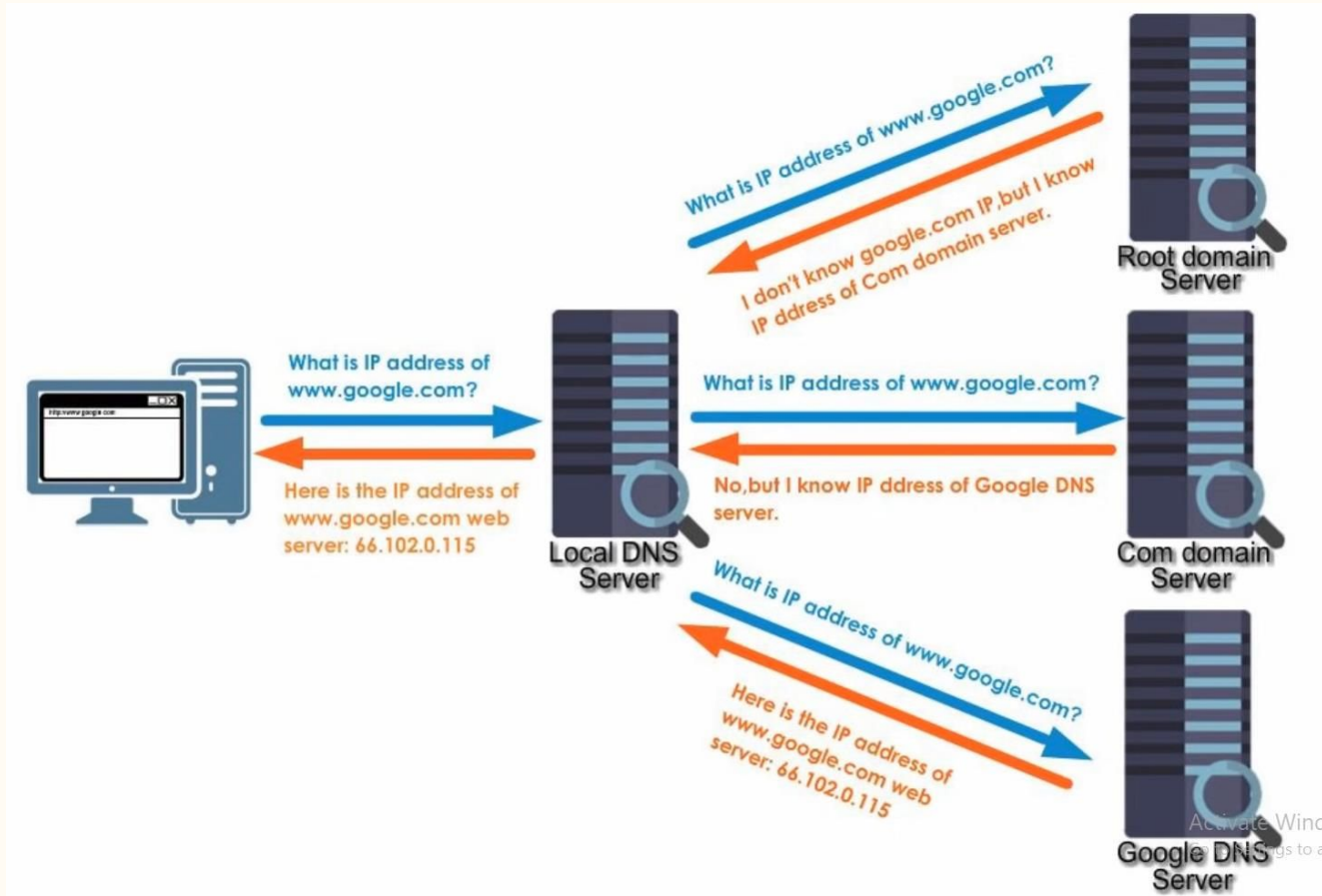
The Recursive Resolver: This is usually provided by your ISP. It acts as the "librarian" that goes out and finds the information for you.

Root Nameserver: The first stop for the resolver. It doesn't know the IP, but it knows which TLD server (like .com) to send the resolver to.

TLD Nameserver: This server knows the location of the Authoritative Nameserver for the specific domain (e.g., the servers belonging to Google).

Authoritative Nameserver: This is the final source of truth. It contains the actual IP address for the requested domain.

Application Layer Protocols: DNS



Application Layer Protocols: DNS

DNS Record Types:

A DNS server doesn't just store IP addresses; it stores different types of "records" depending on the service required:

A Record: Maps a domain name to an IPv4 address.

AAAA Record: Maps a domain name to an IPv6 address.

CNAME (Canonical Name): Maps one domain name to another (an alias).

MX (Mail Exchange): Directs email to a specific mail server.

NS (Name Server): Indicates which servers are authoritative for a zone.

Application Layer Protocols: DNS

DNS Caching:

To prevent the network from being flooded with the same requests, DNS uses Caching.

Browser/OS Cache: Your computer remembers the IP of a site you recently visited.

Resolver Cache: The ISP's resolver stores popular addresses so it doesn't have to go to the Root server every time.

TTL (Time to Live): This is a value in the DNS record that tells the cache exactly how long (in seconds) it should keep the information before asking for a fresh copy.

Application Layer Protocols: DHCP

DHCP is an essential Application Layer protocol used to automate the configuration of devices on IP network

In a manual networking environment, every device requires a unique IP address, subnet mask, and default gateway to communicate.

Components of DHCP

- 1. DHCP Server:** It is typically a server or a router that holds the network configuration information.
- 2. DHCP Client:** It is the endpoint that gets the configuration information from the server like any computer or mobile.
- 3. DHCP Relay Agent:** If you have only one DHCP Server for multiple LAN's then the DHCP relay agent present in every network will forward the DHCP request to the servers. This because the DHCP packets cannot travel across the router. Hence, the relay agent is required so that DHCP servers can handle the request from all the networks.

Application Layer Protocols: DHCP

4. IP address pool: It contains the list of IP address which are available for assignment to the client.

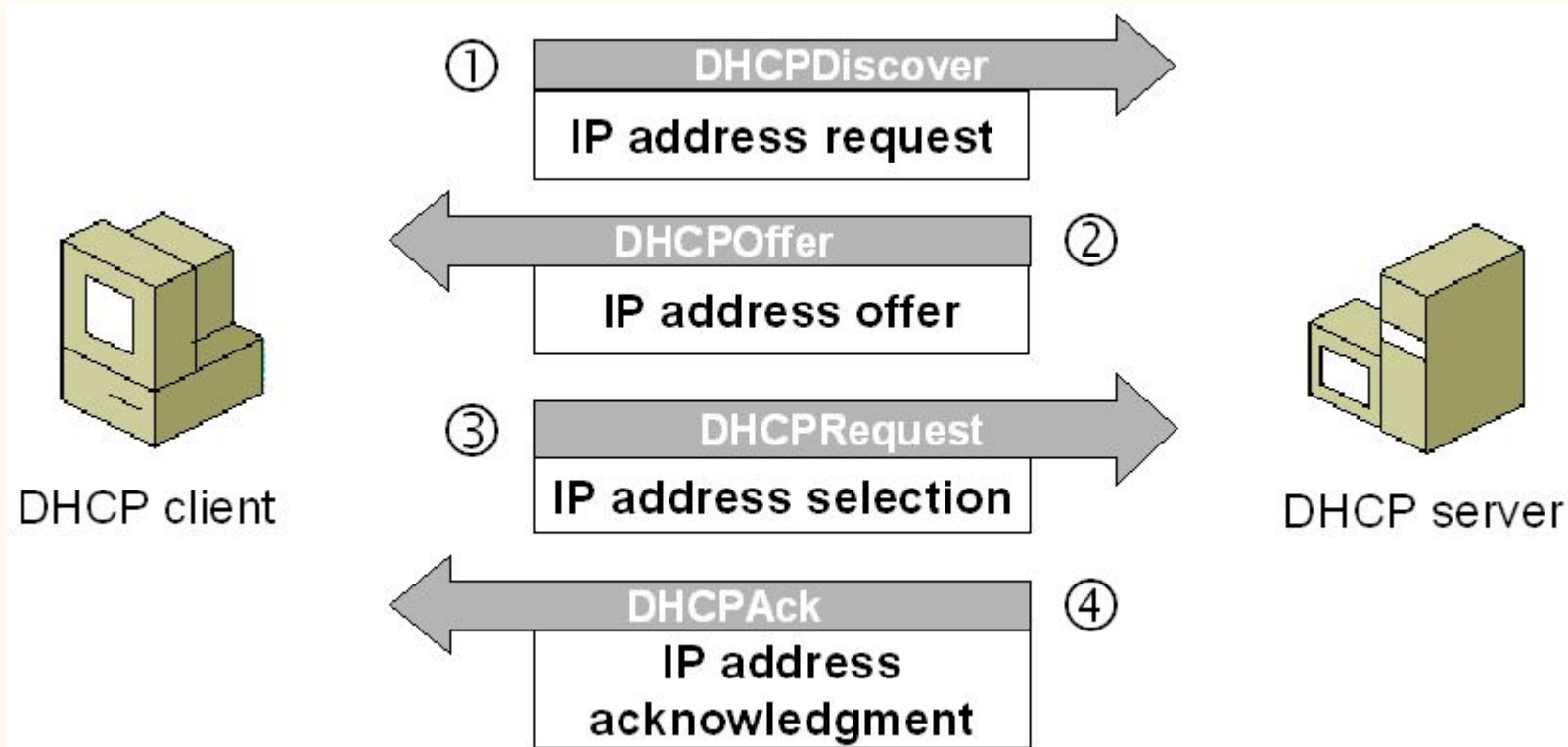
5. Subnet Mask: It tells the host that in which network it is currently present.

6. Lease Time: It is the amount of time for which the IP address is available to the client. After this time the client must renew the IP address.

7. Gateway Address: The gateway address lets the host know where the gateway is to connect to the internet.

Application Layer Protocols: DHCP Working

The communication between a DHCP client (your laptop/phone) and the DHCP server follows a specific four-step handshake known as the DORA process.



Application Layer Protocols: DHCP Working

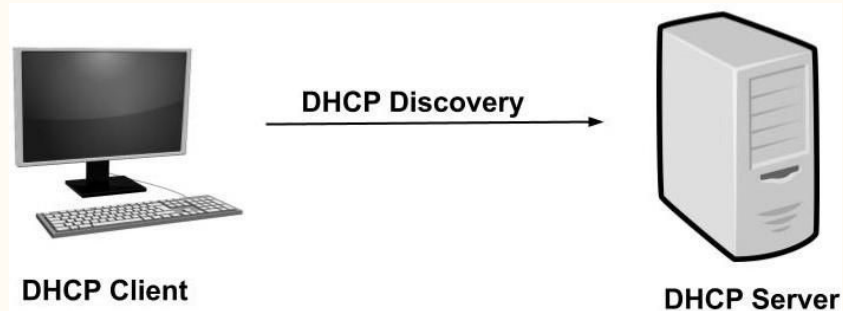
1. Discover (Client → Server):

When a device (like your laptop) connects to a network, it sends a **DHCPDISCOVER** message. It is to find any available DHCP servers on the local network.

Source IP: 0.0.0.0 (Since the client doesn't have an address yet).

Destination IP: 255.255.255.255 (A limited broadcast address that reaches every device on the subnet).

The packet contains the client's MAC Address so the server knows which physical device made the request.



Application Layer Protocols: DHCP Working

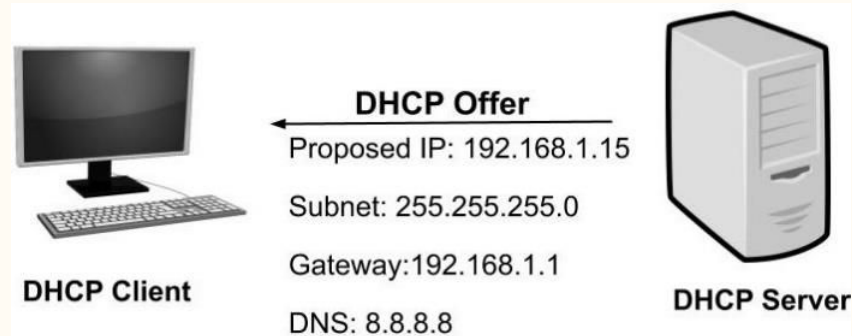
2. Offer (Server → Client):

Any DHCP server that receives the discovery message responds with a **DHCP OFFER**.

The offer includes:

- The proposed IP Address (e.g., 192.168.1.50).
- The Subnet Mask and Default Gateway.
- The Lease Time (how long the client can keep the IP).
- The server's own IP address.

This is also typically sent as a broadcast because the client still hasn't "officially" adopted an IP address.



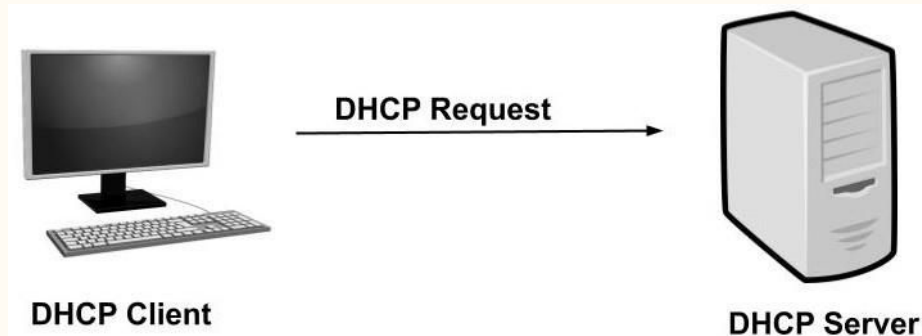
Application Layer Protocols: DHCP Working

3. Request (Client → Server):

The client receives one or more offers (if there are multiple servers). It usually picks the first one it receives and sends a ***DHCPREQUEST***.

Even though the client is talking to a specific server, it broadcasts this message so that all other DHCP servers on the network see the request and can release the IP addresses they were holding in their "Offer" phase.

It includes the server ID of the chosen DHCP server.



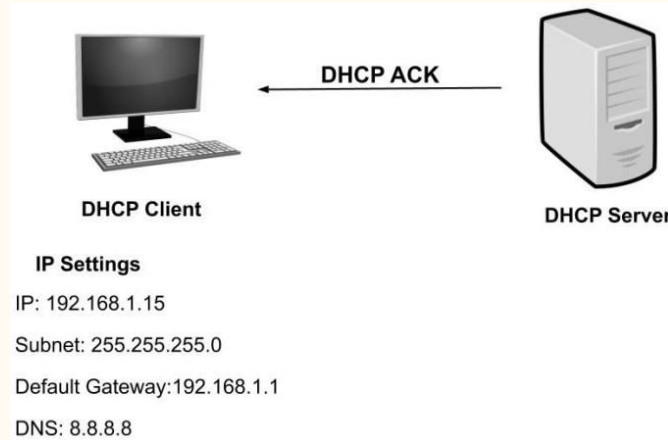
Application Layer Protocols: DHCP Working

4. Acknowledgment (Server → Client)

The chosen server receives the request and sends a ***DHCPACK***

This packet contains the final configuration parameters, including DNS Server addresses and any other network options.

Once the client receives the ACK, it configures its network interface with the new IP. It is now "on the network."



Application Layer Protocols: Telnet

Teletype Network one of the oldest Application Layer protocols, developed in the late 1960s. It was designed to provide a standard method for a user at one site to establish a bidirectional, interactive text-oriented communication session with a remote host.

A network administrator can access the device by telnetting to the IP address or hostname of a remote device. The network administrator will then be presented with a virtual terminal that can interact with the remote host.

It relies on TCP to ensure that the stream of characters is delivered reliably and in the correct order and runs on port 23.

The user runs the Telnet Client software on their local machine, which connects to the Telnet Server (often called a "daemon" or telnetd) listening on the remote host.

Application Layer Protocols: Telnet

In modern networking, Telnet is considered highly insecure and is rarely used over public networks like the Internet.

No Encryption: Telnet sends all data—including usernames and passwords—in plain text. Anyone with a packet sniffer (like Wireshark) on the same network can easily see the credentials as they pass through.

No Authentication: The protocol itself does not provide a way to verify that the server you are connecting to is actually who it claims to be.

Because of these security risks, Telnet has been almost entirely replaced by **SSH (Secure Shell)**.

SSH (Port 22): Provides the same functionality as Telnet but adds strong encryption and authentication, ensuring that data is unreadable to anyone intercepting the packets.

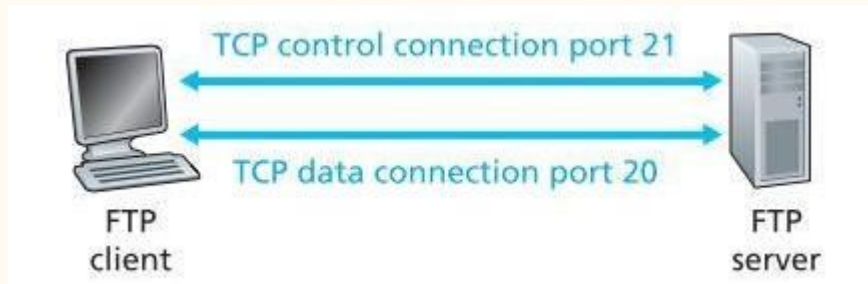
Application Layer Protocols: FTP

File Transfer Protocol (FTP) is the standard mechanism provided by the TCP/IP protocol suite for copying a file from one host to another.

It is specifically designed to handle the complexities of different file systems and directory structures across a network.

FTP allows users to transfer files between two different operating systems (e.g., from a Windows PC to a Linux Server). It handles the translation of data formats so the files remain usable.

Beyond just moving files, FTP allows users to interact with the remote file system to list directories, create new folders, or delete files



Application Layer Protocols: FTP

The Two-Connection Architecture:

FTP is unique at the Application Layer because it establishes two parallel connections between the client and the server. This separation of "instructions" and "data" makes the protocol highly efficient.

A. Control Connection (Port 21)

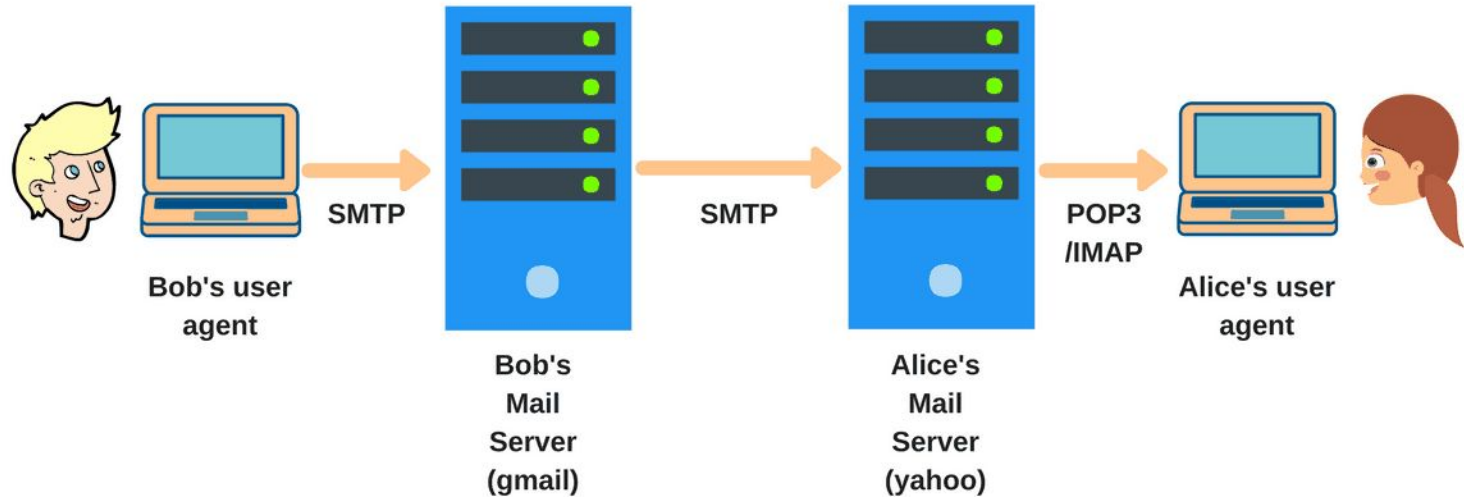
This connection is used for "Control Information."

It carries commands (like USER, PASS, PUT, GET) and server responses (like 220 Service Ready or 230 User logged in). The connection remains open during the entire duration of the FTP session.

B. Data Connection (Port 20)

This connection is used exclusively for the "Data Transfer." It opens only when a file is actually being sent or a directory list is being requested. This is a temporary connection. It opens when the transfer starts and closes immediately once the file transfer is complete.

Application Layer Protocols: E-Mail



Application Layer Protocols: E-Mail

SMTP (Simple Mail Transfer Protocol):

SMTP is the standard protocol used for sending email messages from a client to a server, or between servers.

It acts as a "Push" protocol. It pushes the message from the sender's device to the mail server, and then from the sender's server to the recipient's server.

It is text-based and handles the routing of mail through the Internet using MTA (Mail Transfer Agents).

Port 25: Original port, now primarily used for server-to-server communication.

Port 465: Used for SMTPS (SMTP over SSL).

Limitation: SMTP cannot be used to "pull" or download messages from a server; it is strictly for delivery.

Application Layer Protocols: E-Mail

POP3 (Post Office Protocol version 3)

POP3 is a "Pull" protocol used by an email client to retrieve messages from a mail server.

Ideal for users who access their mail from a single device and want to read messages offline.

Changes made on the local device (like moving an email to a folder) are not reflected on the server or other devices.

Port 110: Standard unencrypted port.

Port 995: Secure POP3 (POP3S) over SSL/TLS.

Application Layer Protocols: E-Mail

IMAP (Internet Message Access Protocol)

IMAP is a more advanced "Pull" protocol for retrieving and managing email messages.

Unlike POP3, IMAP stores the messages on the server and synchronizes them across all devices. When you read an email on your phone, it is marked as "read" on your laptop as well.

It allows for "partial downloads." For example, the client can download just the headers (sender and subject) first and only download the full body or attachments when the user clicks on the mail.

Essential for users who access the same email account from multiple devices (phone, tablet, and PC).

Port 143: Standard unencrypted port.

Port 993: Secure IMAP (IMAPS) over SSL/TLS.

Traffic Analysis: MRTG(Multi Router Traffic Grapher)

MRTG (Multi Router Traffic Grapher):

The Multi Router Traffic Grapher is a free software, originally written in Perl, designed to monitor and graph the traffic on network interfaces.

It was created to answer a fundamental question for network admins: "How much of my link's capacity is being used right now, and how does it compare to last week?"

It generates HTML pages containing GIF or PNG images that provide a visual representation of traffic.

By default, it provides four different views of the network: Daily, Weekly, Monthly, and Yearly.

Traffic Analysis: MRTG(Multi Router Traffic Grapher)

MRTG does not "listen" to packets like a sniffer; instead, it uses a polling mechanism based on the SNMP (Simple Network Management Protocol).

Every five minutes (the default interval), MRTG sends an SNMP GET request to the target device (a router or switch).

It specifically asks for the values of the interface counters, such as ifInOctets (incoming bytes) and ifOutOctets (outgoing bytes).

It calculates the difference between the current counter value and the previous one, divides it by the time interval, and determines the average transfer rate.

It stores this data in its logs and updates the web-accessible graphs immediately.

Traffic Analysis: PRTG(Paessler Router Traffic Grapher)

PRTG is a sophisticated, all-in-one network monitoring solution that evolved from the original concepts of MRTG.

The defining technical feature of PRTG is its Sensor-based system. In PRTG, a "sensor" is the basic monitoring unit.

One sensor typically monitors one measured value on a device (e.g., the traffic of a switch port, the CPU load of a server, or the free space on a disk drive).

Because it uses individual sensors, an administrator can see exactly where a failure occurs. If a web server is slow, PRTG can show whether the bottleneck is the network bandwidth, the RAM usage, or the database response time.

Traffic Analysis: PRTG(Paessler Router Traffic Grapher)

Unlike simpler tools that rely solely on SNMP, PRTG is a multi-protocol engine. It gathers data using a wide array of technical methods:

SNMP: The standard for routers and switches.

WMI (Windows Management Instrumentation): Used for deep monitoring of Windows-based servers and workstations without needing to install agent software.

Flow Technologies (NetFlow, sFlow, J-Flow): Used for high-speed traffic analysis to determine which applications or users are consuming the most bandwidth.

Packet Sniffing: PRTG can intercept and analyze network packets directly to calculate traffic metrics.

Traffic Analysis: SNMP(Simple Network Management Protocol)

SNMP is the standard application-layer protocol used to manage and monitor devices on an IP network.

It provides a common language that allows a central management system to gather information from, and configure, diverse devices such as routers, switches, servers, printers, and IoT devices regardless of the vendor.

Traffic Analysis: SNMP (Simple Network Management Protocol)

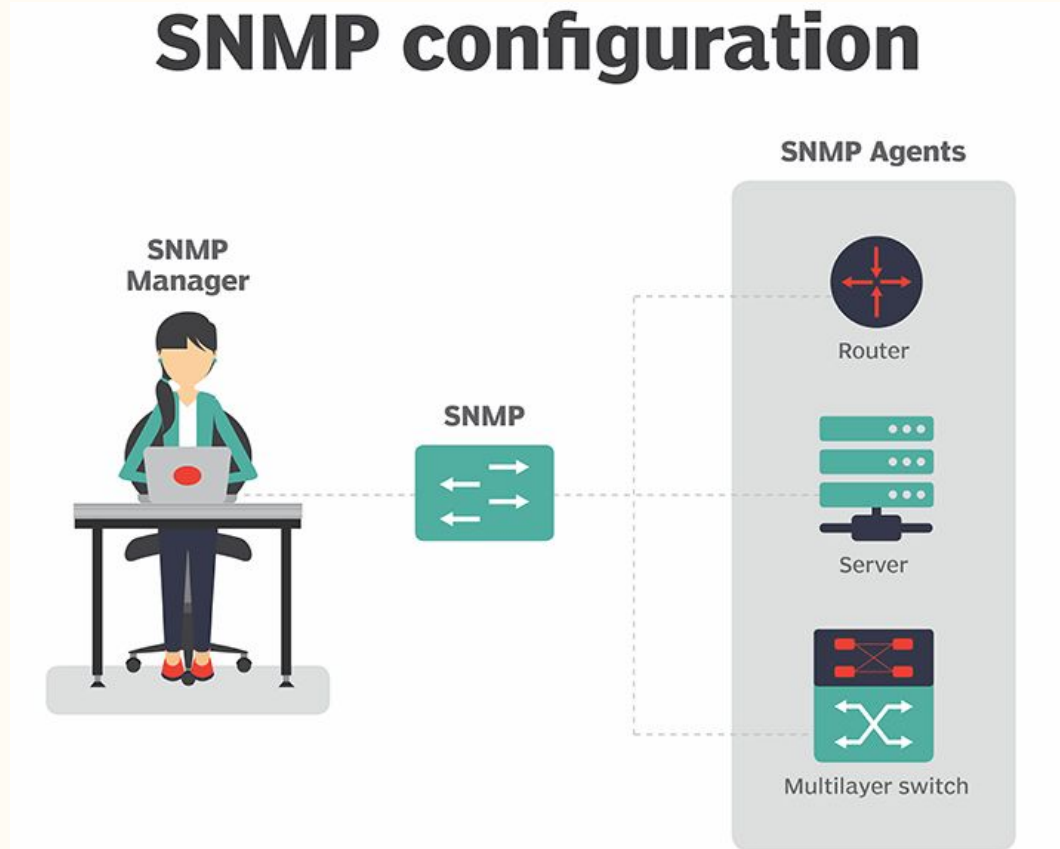
SNMP operates on a Manager/Agent model.

SNMP Manager (NMS): The central software (e.g., PRTG, SolarWinds, Nagios) that requests information from devices. It serves as the "brain" of the monitoring system.

SNMP Agent: A software process running on the managed device (e.g., a Cisco router). It maintains a database of local variables and responds to queries from the Manager.

Management Information Base (MIB): A hierarchical database schema that defines every object an agent can report on. Each object has a unique OID (Object Identifier), which is a dot-separated string of numbers (e.g., 1.3.6.1.2.1.1.5 for system hostname).

Traffic Analysis: SNMP (Simple Network Management Protocol)



Traffic Analysis: SNMP (Simple Network Management Protocol)

Simple Network Management Protocol (SNMP) Operations

GetRequest: The Manager asks the Agent for the value of a specific OID.

GetNextRequest: Used to traverse the MIB table (e.g., reading a list of all interfaces on a switch).

SetRequest: The Manager instructs the Agent to change a configuration value (e.g., shutting down a specific port).

Response: The message sent by the Agent back to the Manager containing the requested data or confirmation of a setting change.

Trap: Critical Feature. Unlike the others, this is an unsolicited message. If a device has a critical failure (like a power supply error), it sends a "Trap" to the Manager immediately without waiting to be polled.

Packet Tracer

Packet Tracer is a powerful visual simulation tool developed by Cisco. It is designed for learning, prototyping, and testing network designs in a risk-free virtual environment.

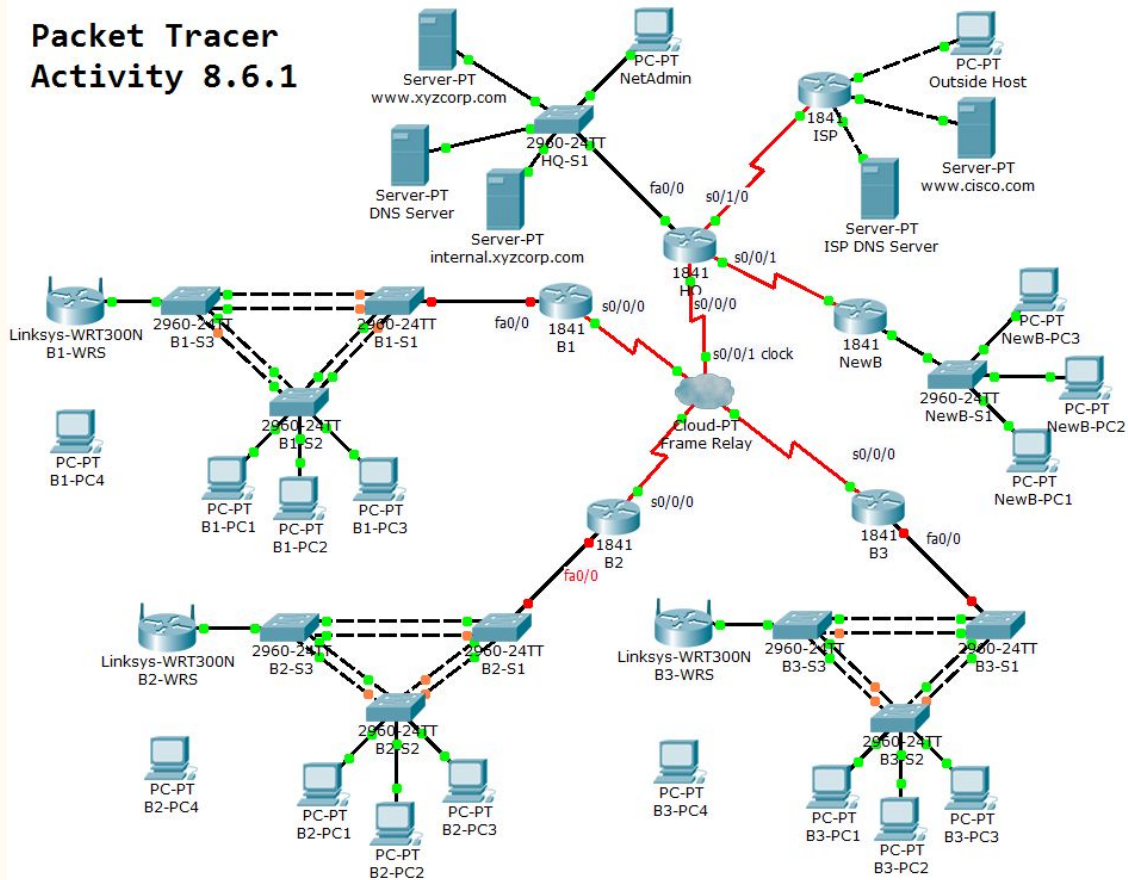
It allows you to build a topology, assign IP addresses, and configure routing protocols (like OSPF or EIGRP) without needing physical hardware.

Environments:

- *Logical Workspace:* Where you build the network, drag and drop routers/switches, and draw cabling connections.
- *Physical Workspace:* Where you can place devices into virtual "racks" and "rooms" to understand the physical realities of cabling and hardware placement.

Packet Tracer

Packet Tracer Activity 8.6.1



Wireshark

Wireshark is the industry-standard network protocol analyzer. Unlike Packet Tracer (which simulates networks), Wireshark captures actual traffic from a real network interface.

At its core, Wireshark performs Deep Packet Inspection (DPI). It captures individual units of data (called packets or frames) moving across your network interface in real-time.

Capture: It intercepts traffic as it passes through your Network Interface Card (NIC).

Dissection: It uses "dissectors" to break down complex binary data into logical fields (e.g., separating the source IP address from the actual message content).

Visualization: It presents this data in a structured, color-coded interface, allowing you to see the conversation between two devices from the first "hello" (handshake) to the final "goodbye" (connection termination).

Wireshark

Wireshark organizes data into three distinct views to make complex data readable:

Packet List Pane: A top-level summary of every packet captured, showing the timestamp, source/destination, and the protocol used.

Packet Details Pane: The "Dissector" view. It displays the data layer-by-layer, allowing you to drill down into the OSI model, from physical cabling details up to application-level data.

Packet Bytes Pane: The raw hexadecimal and ASCII representation of the packet bits, used for advanced forensics and verifying the exact binary structure of the data.

Wireshark

tv-netflix-problems-2011-07-06.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/> Expression...

No.	Time	Source	Destination	Protocol	Length	Info
343	65.142415	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519346 TSecr=551811827
344	65.142715	192.168.0.21	174.129.249.228	HTTP	253	GET /clients/netflix/flash/application.swf?flash_version=flash_lite_2.1&v=1.5&n...
345	65.230738	174.129.249.228	192.168.0.21	TCP	66	80 → 40555 [ACK] Seq=1 Ack=188 Win=6864 Len=0 TSval=551811850 TSecr=491519347
346	65.240742	174.129.249.228	192.168.0.21	HTTP	828	HTTP/1.1 302 Moved Temporarily
347	65.241592	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=188 Ack=763 Win=7424 Len=0 TSval=491519446 TSecr=551811852
348	65.242532	192.168.0.21	192.168.0.1	DNS	77	Standard query 0x2188 A cdn-0.nflximg.com
349	65.276870	192.168.0.1	192.168.0.21	DNS	489	Standard query response 0x2188 A cdn-0.nflximg.com CNAME images.netflix.com.edge...
350	65.277992	192.168.0.21	63.80.242.48	TCP	74	37063 → 80 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=491519482 TSecr=...
351	65.297757	63.80.242.48	192.168.0.21	TCP	74	80 → 37063 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM=1 TSval=3295...
352	65.298396	192.168.0.21	63.80.242.48	TCP	66	37063 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519502 TSecr=3295534130
353	65.298687	192.168.0.21	63.80.242.48	HTTP	153	GET /us/nrd/clients/flash/814540.bun HTTP/1.1
354	65.318730	63.80.242.48	192.168.0.21	TCP	66	80 → 37063 [ACK] Seq=1 Ack=88 Win=5792 Len=0 TSval=3295534151 TSecr=491519503
355	65.321733	63.80.242.48	192.168.0.21	TCP	1514	[TCP segment of a reassembled PDU]

< >

> Frame 349: 489 bytes on wire (3912 bits), 489 bytes captured (3912 bits)

> Ethernet II, Src: Globalsec_00:3b:0a (f0:ad:4e:00:3b:0a), Dst: Vizio_14:8a:e1 (00:19:9d:14:8a:e1)

> Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.21

> User Datagram Protocol, Src Port: 53 (53), Dst Port: 34036 (34036)

> Domain Name System (response)

 [Request In: 348]

 [Time: 0.034338000 seconds]

 Transaction ID: 0x2188

> Flags: 0x8180 Standard query response, No error

 Questions: 1

 Answer RRs: 4

 Authority RRs: 9

 Additional RRs: 9

> Queries

 > cdn-0.nflximg.com: type A, class IN

 > Answers

 > Authoritative nameservers

0020 00 15 00 35 84 f4 01 c7 83 3f 21 88 81 00 00 01 ...5....?|....

0030 00 04 00 09 00 09 05 63 64 6e 2d 30 07 6e 66 6cc dn-0.nfl

0040 78 69 6d 67 03 63 6f 6d 00 00 01 00 01 c8 0c 00 ximg.com

0050 05 00 01 00 00 05 29 00 22 06 69 6d 61 67 65 73). ".images

0060 07 6e 65 74 66 6c 69 78 03 63 6f 6d 09 65 64 67 .netflix .com.edge

0070 65 73 75 69 74 65 03 6e 65 74 00 c0 2f 00 05 00 esuite.n et../...

Identification of transaction (dns.id), 2 bytes

Packets: 10299 · Displayed: 10299 (100.0%) · Load time: 0:0.182 | Profile: Default

The End